

Metaparameters and metaoptimization

Evolutionary structure selection algorithms

MIPT

2026

Hyperparameters

Definition

Prior for parameters $\mathbf{w}$ and structure $\mathbf{\Gamma}$ of the model $\mathbf{f}$ is a distribution $p(\mathbf{W}, \mathbf{\Gamma} | \mathbf{h}) : \mathbb{W} \times \Gamma \times \mathbb{H} \rightarrow \mathbb{R}^+$, where $\mathbb{W}$ is a parameter space, Γ is a structure space.

Definition

Hyperparameters $\mathbf{h} \in \mathbb{H}$ of the models are the parameters of $p(\mathbf{w}, \mathbf{\Gamma} | \mathbf{h})$ (parameters of prior $\mathbf{f}$).

Metaparameters

Wikipedia

A parameter that controls the value of one or more others.

Definition

Metaparameters λ are the parameters of optimization problem.

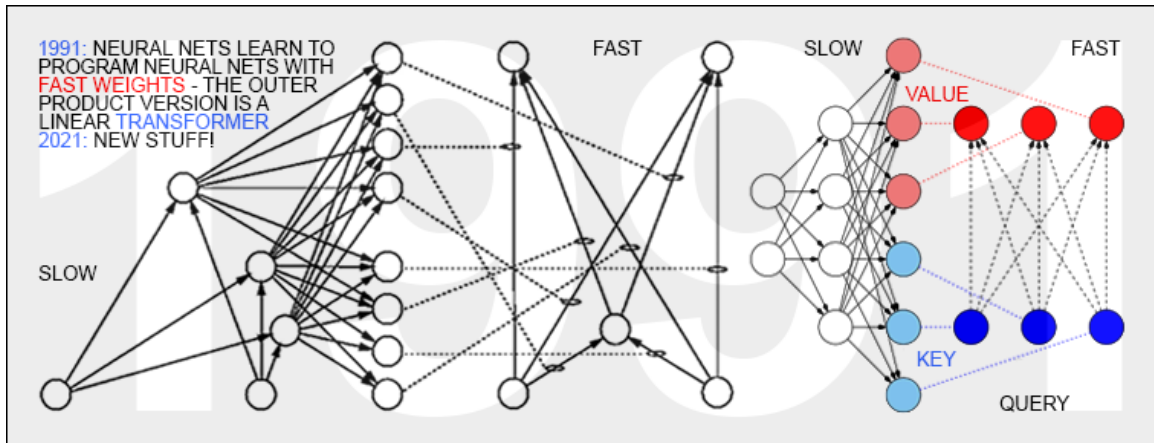
Is dropout rate a metaparameter or hyperparameter?

Consider using Gumbel-Softmax as a variational distribution for model deep neural network structure. Is temperature a metaparameter or hyperparameter?

A neural network that embeds its own meta-levels

A model is decomposed into submodels with different subtasks:

- “Normal” model: optimization and inference.
- Evaluation model: validation performance estimation.
- Analyzing model: analysis of model parameters.
- Modifiyng model: parameter modification.



<https://people.idsia.ch/~juergen/fast-weight-programmer-1991-transformer.html>

In-context learning with attention ¹

Attention

Definition(Unnormalized Dot Attention) let $\mathbf{K} = (\mathbf{k}_1, \dots, \mathbf{k}_T) \in \mathbb{R}^{d_{\text{in}} \times T}$ and $\mathbf{V} = (\mathbf{v}_1, \dots, \mathbf{v}_T) \in \mathbb{R}^{d_{\text{out}} \times T}$ denote matrices representing T key and value vectors. Let $\mathbf{q} \in \mathbb{R}^{d_{\text{in}}}$ denote a query vector. An unnormalised linear dot attention operation $\text{Attention}(\mathbf{K}, \mathbf{V}, \mathbf{q})$ computes the following weighted average of value vectors $\mathbf{v}_t$:

$$\text{Attention}(\mathbf{K}, \mathbf{V}, \mathbf{q}) = \sum_{t=1}^T \alpha_t \mathbf{v}_t \quad (1)$$

where the weights $\alpha_t = \mathbf{k}_t^\top \mathbf{q} \in \mathbb{R}$ are dot products between key $\mathbf{k}_t$ and query $\mathbf{q}$ vectors, and are called attention weights.

¹See Sergey Skorik's talk, spring 2023

Idea

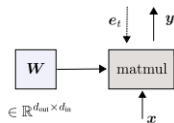


Figure 1. The primal form of a linear layer to be contrasted with the dual form in Figure 2.

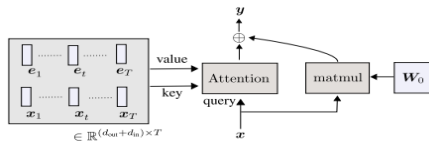
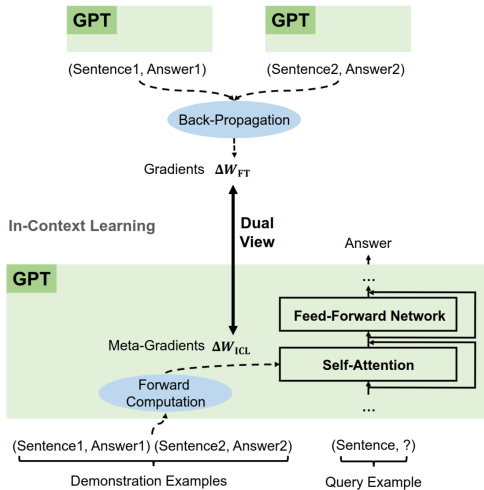


Figure 2. The dual form of a linear layer trained by gradient descent is a key-value memory with attention storing the entire training experience. Compare to the primal form in Figure 1.

Idea

Finetuning



Duality

Corollary

The following systems S_1 , S_2 are equivalent:

- S_1 (*Primal form*): A linear layer in a neural network trained by gradient descent in some error function using T training inputs to this layer $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ with $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$ and corresponding (backpropagation) error signal $(\mathbf{e}_1, \dots, \mathbf{e}_T)$ with $\mathbf{e}_t \in \mathbb{R}^{d_{out}}$ obtained by gradient descent. Its weight matrix $\mathbf{W} \in \mathbb{R}^{d_{in} \times d_{out}}$ is thus:

$$\mathbf{W} = \mathbf{W}_0 + \sum_{t=1}^T \mathbf{e}_t \otimes \mathbf{x}_t \quad (2)$$

Where $\mathbf{W}_0 \in \mathbb{R}^{d_{in} \times d_{out}}$ is the initialisation. The layer transforms input $\mathbf{x} \in \mathbb{R}^{d_{in}}$ to output $S_1(\mathbf{x}) \in \mathbb{R}^{d_{out}}$ as:

$$S_1(\mathbf{x}) = \mathbf{W}\mathbf{x} \quad (3)$$

Corollary

- S_2 (*Dual form*): A layer which stores T key-value pair $(\mathbf{x}_1, \mathbf{e}_1), \dots, (\mathbf{x}_T, \mathbf{e}_T)$ i.e., a key matrix $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_T) \in \mathbb{R}^{d_{\text{in}} \times T}$ and a value matrix $\mathbf{E} = (\mathbf{e}_1, \dots, \mathbf{e}_T) \in \mathbb{R}^{d_{\text{out}} \times T}$, and a weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ which transforms input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $S_2(\mathbf{x}) \in \mathbb{R}^{d_{\text{out}}}$ as:

$$S_2(\mathbf{x}) = \mathbf{W}_0 \mathbf{x} + \text{Attention}(\mathbf{X}, \mathbf{E}, \mathbf{x}) \quad (4)$$

Proof:

$$S_1(\mathbf{x}) = \mathbf{W} \mathbf{x} = \mathbf{W}_0 \mathbf{x} + \sum_{t=1}^T \mathbf{e}_t \otimes \mathbf{x}_t \cdot \mathbf{x} = \mathbf{W}_0 \mathbf{x} + \text{Attention}(\mathbf{X}, \mathbf{E}, \mathbf{x})$$

Remarks

Remark 1

Nothing is "forgotten":

The entire life of an NN is recorded and stored as a key matrix X and value matrix E . Roughly speaking, the only limitation of the model's capability to "remember" something is the limitation of the retrieval process

Remark 2

Unlimited memory size is not necessarily useful:

Systems which store everything in memory by increasing its size for each new event (S_2) are not necessarily better than those with a fixed size storage (S_1).

Remark 3

Non-Uniqueness:

The expression of a linear layer as an attention system is not unique.

Experiment to compare Fine-tuning and ICL

- Fine-tune model using the same examples as in ICL;
- Use the following quality criteria:
 - ▶ Recall to Finetuning Predictions (**Rec2FTP**): percentage of queries that fine-tuning and ICL correctly predicts among queries that fine-tuning predicts correctly, but zero-shot not.
 - ▶ Similarity of Attention Output Updates (**SimAOU**): cosine between updates of output representations after fine-tuning and ICL.
 - ▶ Similarity of Attention Map (**SimAM**): cosine between attention maps before softmax operations.

Attention with momentum

Since we have established a relation of Attention layers with SGD-like optimization, we can plug momentum into attention:

$$\text{MoAttn}(\mathbf{V}, \mathbf{K}, \mathbf{v}) = \text{Attention}(\mathbf{V}, \mathbf{K}, \mathbf{v}) + \sum_{i=1}^{N-1} \lambda_{EMA}^{N-i} \mathbf{v}_i.$$

Gradient Descent
(GD)



GD with
Momentum



Attention



Momentum
Attention



Attention with momentum: results

Model	Train ₁₀₂₄	Valid ₂₅₆	Valid ₅₁₂	Valid ₁₀₂₄
Transformer	17.61	19.50	16.87	15.14
Transformer _{MoAttn}	17.55	19.37	16.73	15.02

Table 4: Perplexity on the training set and validation sets with different input lengths for language modeling. Applying momentum to attention introduces a consistent perplexity improvement compared with the vanilla Transformer.

Model	SST5	IMDB	MR	CB	ARC-E	PIQA	Average
Transformer	25.3	64.0	61.2	43.9	48.2	68.7	51.9
Transformer _{MoAttn}	27.4	70.3	64.8	46.8	50.0	69.0	54.7

Table 5: Accuracy on six in-context learning tasks. Introducing momentum into attention improves the accuracy of the vanilla Transformer by 2.8 on average.

Learning to learn by gradient descent by gradient descent

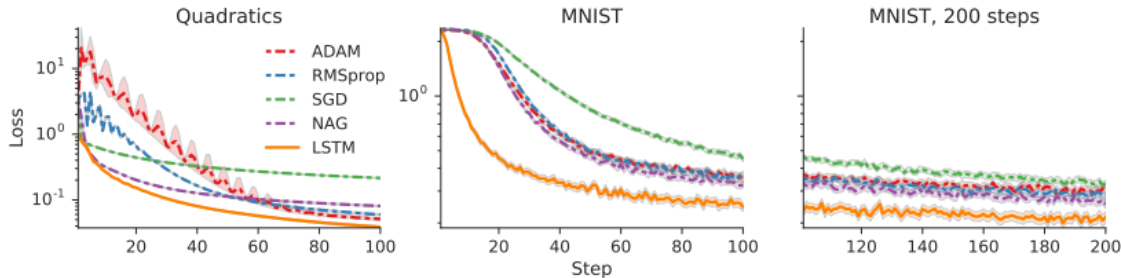
Idea: consider optimization T as a differential function:

$$T(\theta) = \text{LSTM}(\theta).$$

Optimization problem:

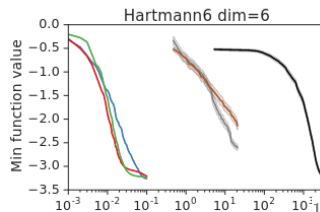
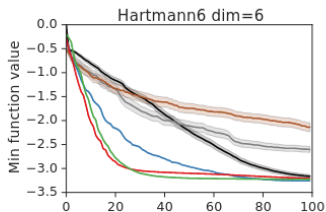
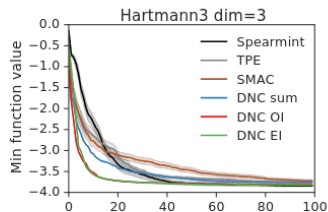
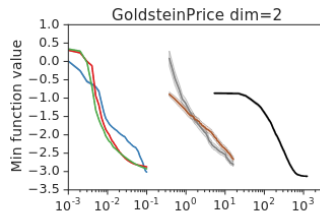
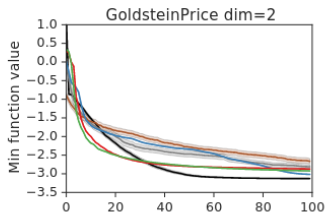
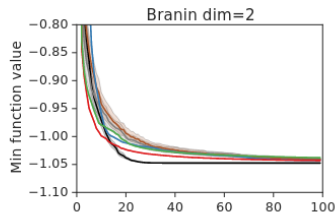
$$\sum_{t=t_0}^{t_\eta} L(T^t(\theta_{t_0})) \rightarrow \max.$$

LSTM has a small number of parameters, it shares parameters for each metaparameters.



Learning to Learn without Gradient Descent by Gradient Descent

- Using the same idea, but for hyperparameters optimization (alternative to GP)
- Can be used with a combination of different optimization functions (e.g. EI)
- Does not require gradient at the «test» time



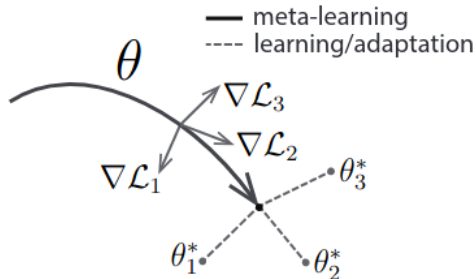
Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks

Algorithm 1 Model-Agnostic Meta-Learning

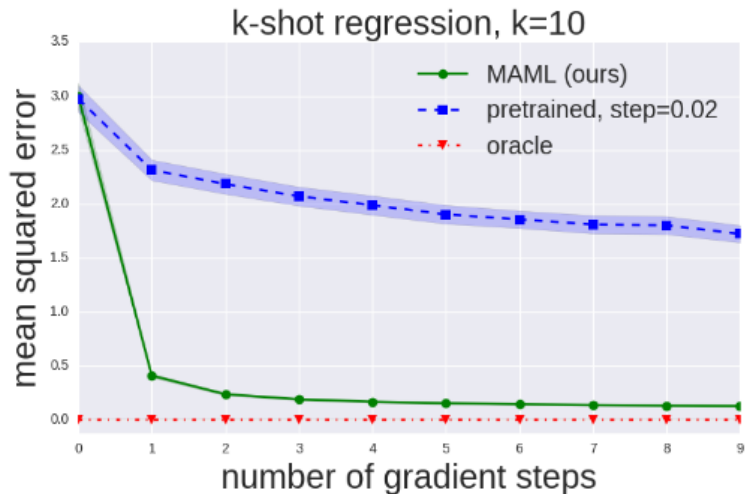
Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

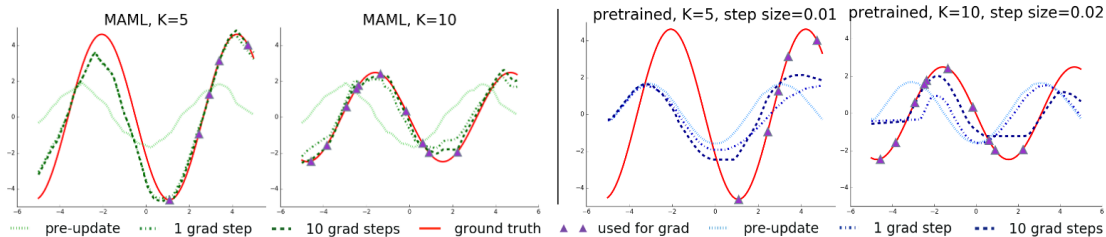
- 1: randomly initialize θ
 - 2: **while** not done **do**
 - 3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
 - 4: **for all** $\mathcal{T}_i$ **do**
 - 5: Evaluate $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ with respect to K examples
 - 6: Compute adapted parameters with gradient descent: $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$
 - 7: **end for**
 - 8: Update $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$
 - 9: **end while**
-



Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks



Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks



Hypernetworks

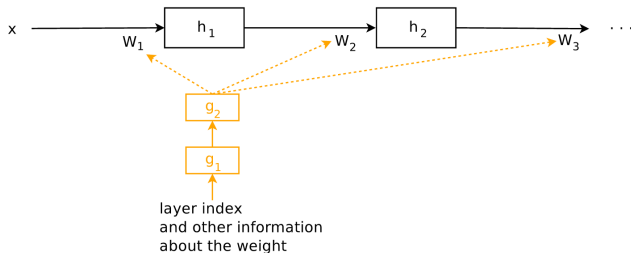
Definition

Given a set Λ .

Hypernetwork is a parametric mapping from Λ to set $\mathbb{R}^n$ of the model $\mathbf{f}$ parameters:

$$G : \Lambda \times \mathbb{R}^u \rightarrow \mathbb{R}^n,$$

where $\mathbb{R}^u$ is a set of hypernetwork parameters.

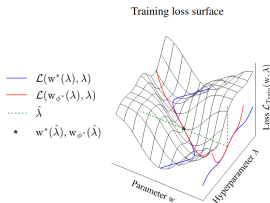


Stochastic Hyperparameter Optimization through Hypernetworks

$$E_{\lambda} \left(-\log p(\mathcal{D}|\mathbf{w}(\lambda)) + \lambda \|\mathbf{w}(\lambda)\|_2^2 \right) \rightarrow \min$$

Theorem

Sufficiently powerful hypernetworks can learn continuous best-response functions, which minimizes the expected loss for all hyperparameter distributions with convex support.



Lorraine et al., 2016

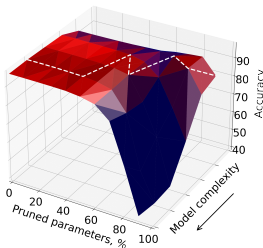
Deep learning model selection with parametric complexity control

Theorem (Grebenkova, 2023)

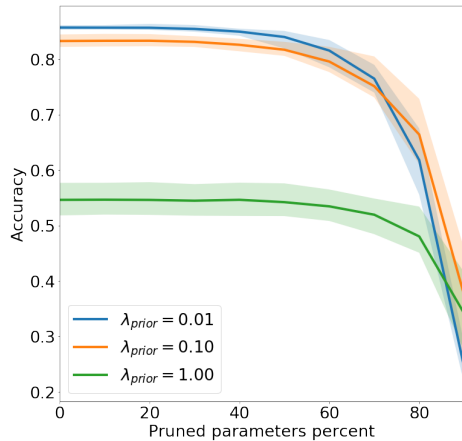
Hypernetwork allows to learn not only the best-response functions, but also the statistical properties of the models.

Corollary

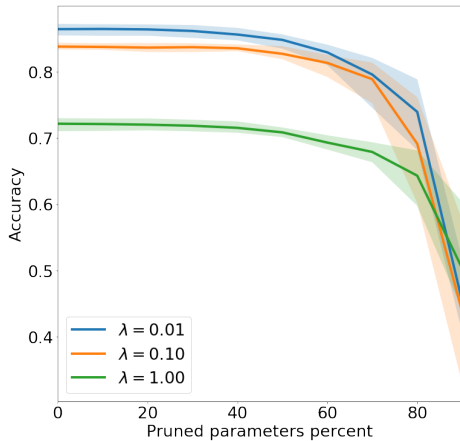
We can control the metaparameters, such as pruning rate or model complexity.



Example: CIFAR-10



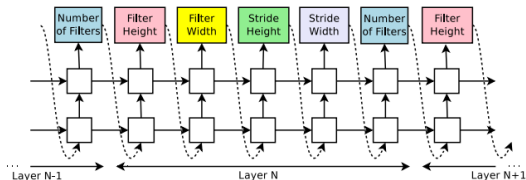
CNN



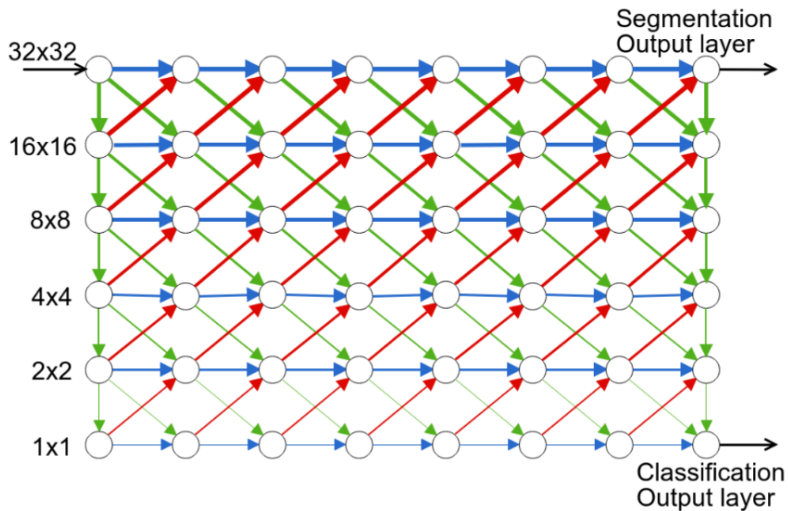
CNN with hypernetwork

Neural Architecture Search with Reinforcement Learning

The structure is selected using controllet.
The optimization of model parameters is conducted in a loop of structure selection.



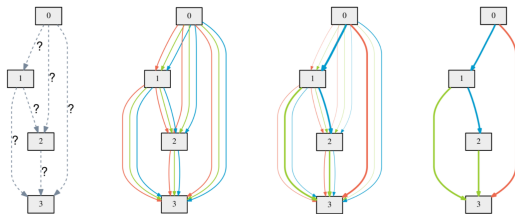
Supernetworks



DARTS

The model is a multigraph, where edges $[g^e]$ correspond to submodels, vertices $\mathbf{f}_v(\mathbf{x})$ are the results of submodels:

$$\mathbf{f}_v = \langle \gamma, \text{softmax}([g^e(\mathbf{x}))] \rangle.$$



ENAS

Two-stage optimization:

- ① Parameter optimization
 - ▶ Fix controller parameters
 - ▶ At each optimization step sample a structure Γ
 - ▶ Optimize model parameters: $\mathbf{w} = \arg \min L$
- ② Controller optimization
 - ▶ Fix model parameters $\mathbf{w}$
 - ▶ Optimize controller parameters.

ENAS vs DARTS

Architecture	Test Error (%)	Params (M)	Search Cost (GPU days)	#ops	Search Method
DenseNet-BC (Huang et al., 2017)	3.46	25.6	–	–	manual
NASNet-A + cutout (Zoph et al., 2018)	2.65	3.3	2000	13	RL
NASNet-A + cutout (Zoph et al., 2018) [†]	2.83	3.1	2000	13	RL
BlockQNN (Zhong et al., 2018)	3.54	39.8	96	8	RL
AmoebaNet-A (Real et al., 2018)	3.34 ± 0.06	3.2	3150	19	evolution
AmoebaNet-A + cutout (Real et al., 2018) [†]	3.12	3.1	3150	19	evolution
AmoebaNet-B + cutout (Real et al., 2018)	2.55 ± 0.05	2.8	3150	19	evolution
Hierarchical evolution (Liu et al., 2018b)	3.75 ± 0.12	15.7	300	6	evolution
PNAS (Liu et al., 2018a)	3.41 ± 0.09	3.2	225	8	SMBO
ENAS + cutout (Pham et al., 2018b)	2.89	4.6	0.5	6	RL
ENAS + cutout (Pham et al., 2018b) [*]	2.91	4.2	4	6	RL
Random search baseline [‡] + cutout	3.29 ± 0.15	3.2	4	7	random
DARTS (first order) + cutout	3.00 ± 0.14	3.3	1.5	7	gradient-based
DARTS (second order) + cutout	2.76 ± 0.09	3.3	4	7	gradient-based

Why does stochasticity work?

Scheme is a genotype or a subset of genotypes, which are encoded by binary string with mask in the format [001 * 1]. Mask symbol * tells us that the corresponding feature is not important in this genotype.

Holland's schema theorem

$$N(h, t + 1) \geq N(h, t) \frac{Q(h, t)}{E_h Q(h, t)} (1 - p),$$

where h is a scheme, N is an amount of schemes, p is a probability of scheme destroy w.r.t. genetic operators, Q is a quality.

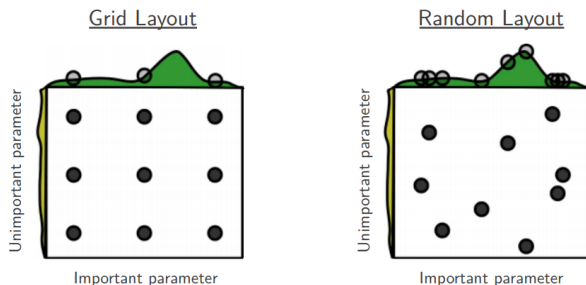
- Schemes with larger mask are more viable;
- Schemes with **larger mask** and **quality better than average** are more viable.

Naive structure search

Variants:

- Grid search;
- Random search;

Both methods suffer from curse of dimensionality. The random search is more effective if the search space is degenerate.



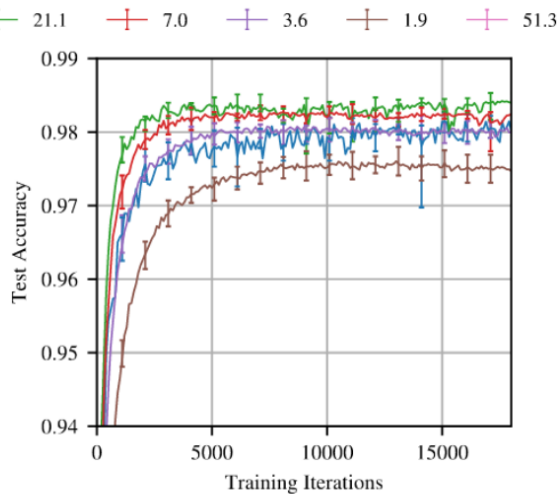
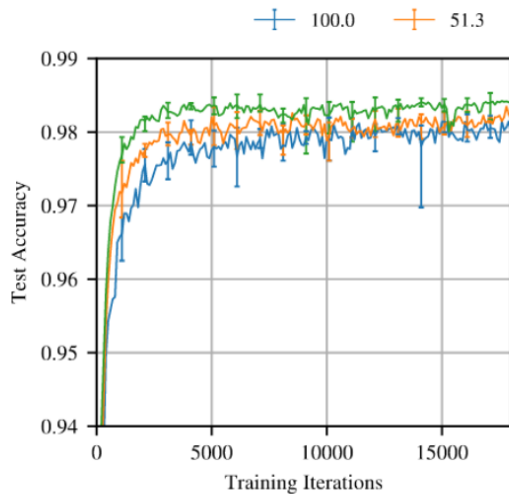
Bergstra et al., 2012

Lottery ticket

The Lottery Ticket Hypothesis, Frankle et al., 2019

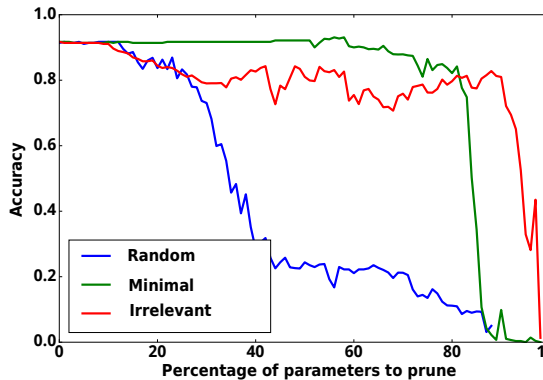
A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.

Lottery ticket: MNIST

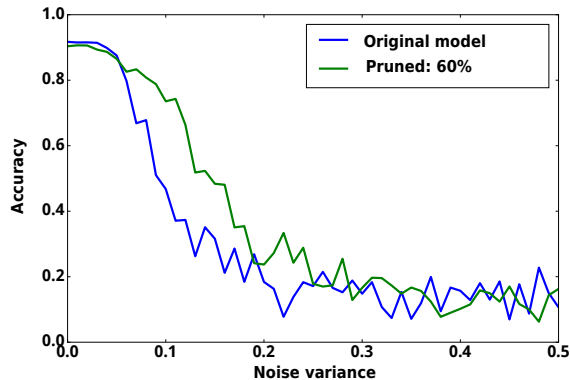


Model structure selection challenge

Data likelihood does not change with removing redundant parameters.



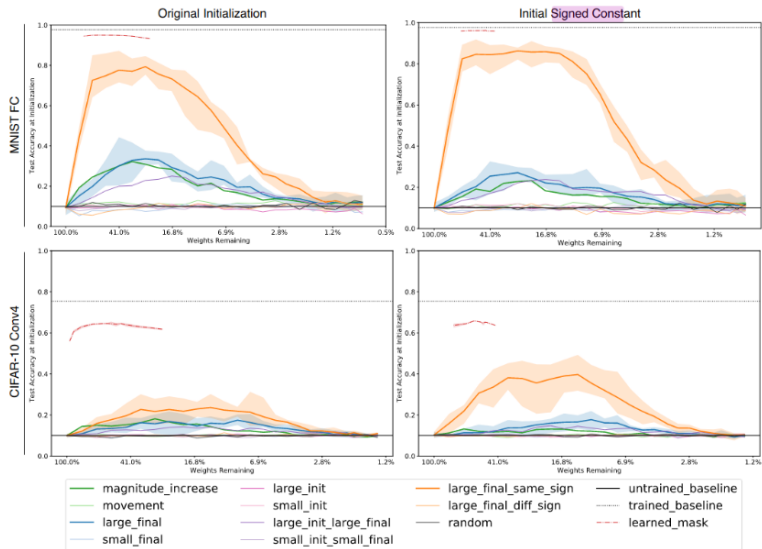
Redundancy of model parameters



Model robustness

Deep learning models have implicitly redundant complexity.

Lottery ticket, results



Weight agnostic NN, Gaier et al., 2019

Model $\mathbf{f}$ is a MLP with constant parameters and a structure $\mathbf{\Gamma}$ that assigns connections between neurons and activations.

No backprop, only evolutionary algorithms!

WANN

1.) Initialize

Create population of minimal networks.

2.) Evaluate

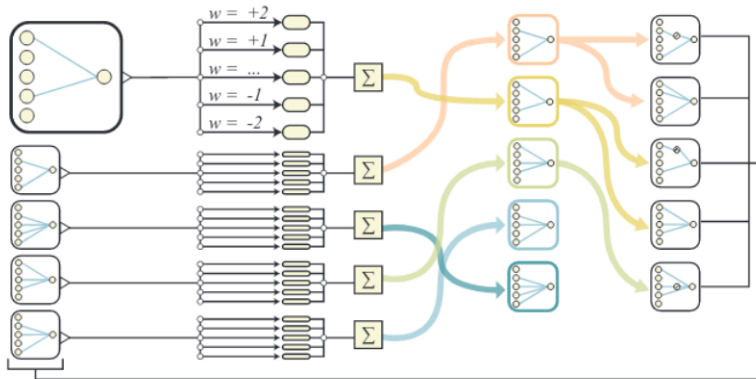
Test with range of shared weight values.

3.) Rank

Rank by performance and complexity

4.) Vary

Create new population by varying best networks.



WANN

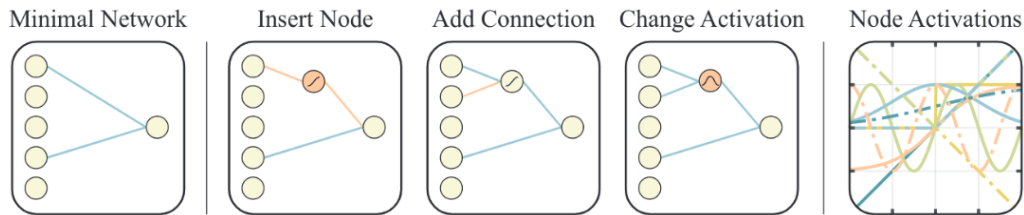


Figure 3: *Operators for Searching the Space of Network Topologies*

Left: A minimal network topology, with input and outputs only partially connected.

Middle: Networks are altered in one of three ways. *Insert Node:* a new node is inserted by splitting an existing connection. *Add Connection:* a new connection is added by connecting two previously unconnected nodes. *Change Activation:* the activation function of a hidden node is reassigned.

Right: Possible activation functions (linear, step, sin, cosine, Gaussian, tanh, sigmoid, absolute value, invert (i.e. negative linear), ReLU) shown over the range $[2, 2]$.

WANN

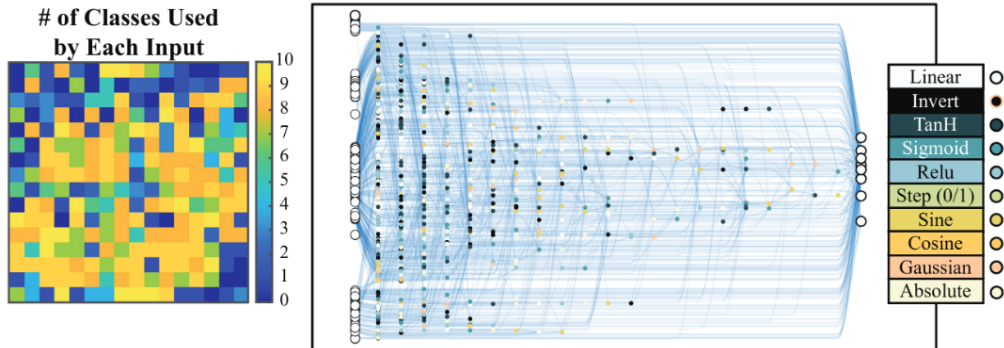


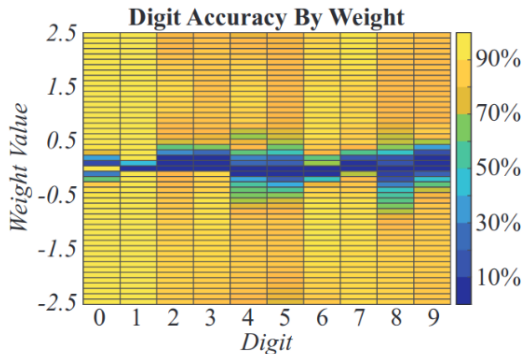
Figure 7: *MNIST classifier network (1849 connections)*

WANN

WANN	Test Accuracy
Random Weight	82.0% $\pm$ 18.7%
Ensemble Weights	91.6%
Tuned Weight	91.9%
Trained Weights	94.2%

ANN	Test Accuracy
Linear Regression	91.6% [62]
Two-Layer CNN	99.3% [15]

Figure 6: *Classification Accuracy on MNIST.*



Diffusion is evolution

Idea: higher *fitness* $\iff$ higher *pdf*.

- ① Fitness function: $f : \mathbb{R}^b \rightarrow \mathbb{R}$
- ② mapping into pdf g : $g[f(x)] = p(x)$. Then $p(x_0 = x) = g[f(x)]$
- ③ In DDPM, the error ε is estimated using neural network θ :

$$\varepsilon = \frac{x_t - \sqrt{\alpha_t}x_0}{\sqrt{1 - \alpha_t}} = \varepsilon_\theta(x_t, t)$$

- ④ In contrast with DDPM, DiffEvo estimates the x_0 from $(x_t, f(x_t))$:

$$p(x_0 = x | x_t) = [\text{Bayes}] = \frac{p(x_t | x_0 = x)g[f(x)]}{p(x_t)}$$

Diffusion is evolution

- 1 In contrast with DDPM, DiffEvo estimates the x_0 from $(x_t, f(x_t))$:

$$p(x_0 = x | x_t) = [\text{Bayes}] = \frac{p(x_t | x_0 = x) g[f(x)]}{p(x_t)}$$

- 2 $p(x_t | x_0 = x) \sim \mathcal{N}(x_t | \sqrt{\alpha_t}x; 1 - \alpha_t)$ — is known
- 3 As our loss is MSE, the estimated x_0 should be the weighted average of the sample x :

$$\hat{x}_0(x_t, \alpha, t) = \sum_{x \sim p_{\text{eval}}(x)} g[f(x)] \frac{\mathcal{N}(x_t | \sqrt{\alpha_t}x; 1 - \alpha_t)}{p(x_t)} \cdot x$$

Idea: by substituting the above formula for the x_0 estimation into diffusion sampling, we get Evolution Diffusion algorithm.

Experiments

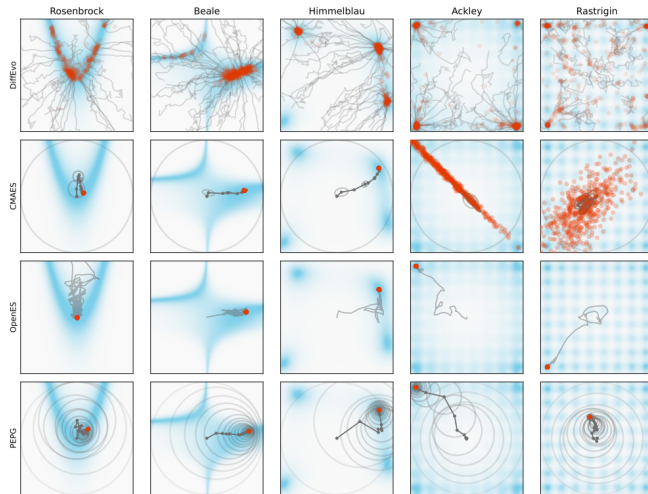


Рис.: Comparison with other algorithms

Why symbolic models for AutoML?²

- Classical approaches for AutoML usually work in a very limited search space.
- **What if we don't have any knowledge about the model to use?**

²See Kseniia Petrushina's talk, 2023

Proposed approach

Idea

Evolutionary symbolic search over basic mathematical operations

Process formulation

- ① Given a set of ML tasks $\mathcal{T}$, find optimal algorithm $\alpha^* \in \mathcal{A}$
- ② Quality of α is measured on $\mathcal{T}_{search} \subset \mathcal{T}$. Each search experiment produces candidate algorithm
- ③ The best candidate is chosen on $\mathcal{T}_{select} \subset \mathcal{T}$

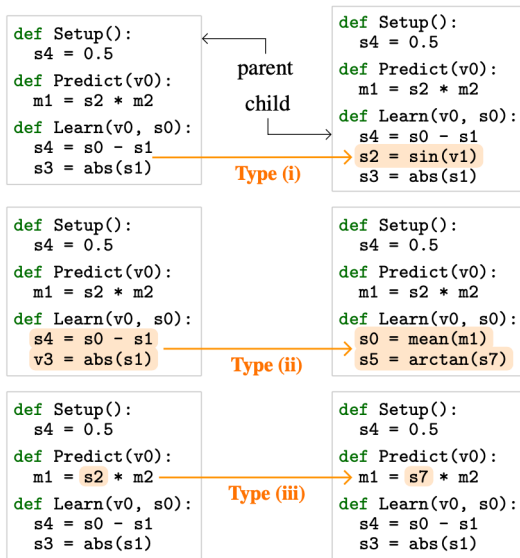
Search space

- The model is decomposed into 3 functions:
 - ▶ Setup: parameters initialization
 - ▶ Predict: make a prediction
 - ▶ Learn: make adjustment in prediction
- Each program has special memory registers for vectors and scalars
- Each program line is constructed by basic arithmetic operation and one or multiple registers
- The objects, labels and predicted labels are stored in the pre-defined registers (v0, s0, s1)

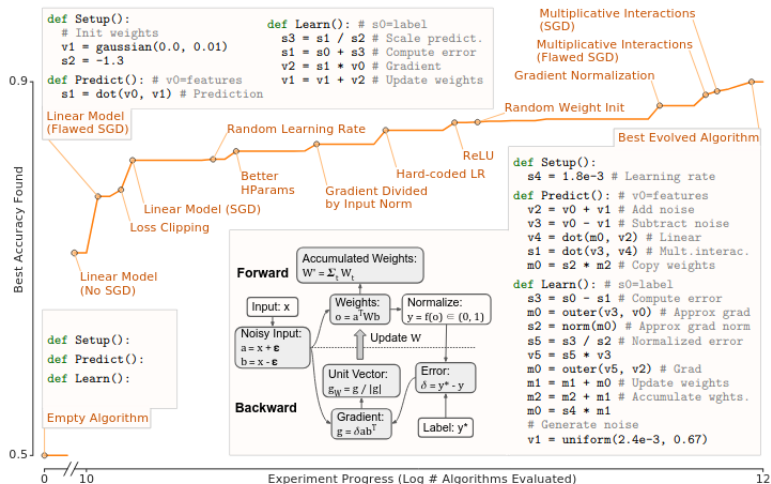
```
# (Setup, Predict, Learn) = input ML algorithm.
# Dtrain / Dvalid = training / validation set.
# sX/vX/mX: scalar/vector/matrix var at address X.
def Evaluate(Setup, Predict, Learn, Dtrain,
Dvalid):
    # Zero-initialize all the variables (sX/vX/mX).
    initialize_memory()
    Setup() # Execute setup instructions.
    for (x, y) in Dtrain:
        v0 = x # x will now be accessible to Predict.
        Predict() # Execute prediction instructions.
        # s1 will now be used as the prediction.
        s1 = Normalize(s1) # Normalize the prediction.
        s0 = y # y will now be accessible to Learn.
        Learn() # Execute learning instructions.

    sum_loss = 0.0
    for (x, y) in Dvalid:
        v0 = x
        Predict() # Only Predict(), not Learn().
        s1 = Normalize(s1)
        sum_loss += Loss(y, s1)
    mean_loss = sum_loss / len(Dvalid)
    # Use validation loss to evaluate the algorithm.
    return mean_loss
```

Evolutionary algorithm: mutation examples



Results



Relative success rate of evolution and random search (RS).

Emerging techniques

Emerging techniques

- Regularization

$$\mathbf{a} = \mathbf{x} + \mathbf{u}, \quad \mathbf{b} = \mathbf{x} - \mathbf{u}, \quad \mathbf{u} = \mathbf{U}(\alpha, \beta)$$

- Multiplicative interactions

$$\mathbf{o} = \mathbf{a}^T \mathbf{W} \mathbf{b}$$

- Gradient normalization

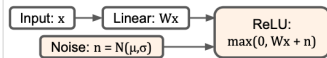
$$\mathbf{g}_w = \frac{\mathbf{g}}{|\mathbf{g}|}; \quad \mathbf{g} = \delta \mathbf{a} \mathbf{b}^T; \quad \delta = \mathbf{y}^* - \mathbf{y}$$

- Weight averaging

$$\mathbf{w}' = \sum_t \mathbf{w}_t$$

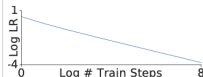
Adaptation to different tasks

```
def Predict():  
    ... # Omitted/irrelevant code  
    # v0=features; m1=weight matrix  
    v6 = dot(m1, v0) # Apply weights  
    # Random vector,  $\mu=-0.5$  and  $\sigma=0.41$   
    v8 = gaussian(-0.5, 0.41)  
    v6 = v6 + v8 # Add it to activations  
    v7 = maximum(v9, v6) # ReLU, v9 $\approx$ 0  
    ...
```



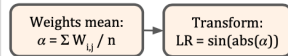
(a) Adaptation to few examples.

```
def Setup():  
    # LR = learning rate  
    s2 = 0.37 # Init. LR  
    ...  
def Learn():  
    # Decay LR  
    s2 = arctan(s2)  
    ...
```



(b) Adaptation to fast training.

```
def Learn():  
    s3 = mean(m1) # m1 is the weights.  
    s3 = abs(s3)  
    s3 = sin(s3)  
    # From here down, s3 is used as  
    # the learning rate.  
    ...
```



(c) Adaptation to multiple classes.

Adaptations to different task types.

References

- Schmidhuber, Jürgen. "A neural network that embeds its own meta-levels." IEEE International Conference on Neural Networks. IEEE, 1993.
- Dougal Maclaurin et. al, Gradient-based Hyperparameter Optimization through Reversible Learning, 2015
- Работы по Learning to learn by gradient descent
- Chen Y. et al. Learning to learn without gradient descent by gradient descent //International Conference on Machine Learning. – PMLR, 2017. – С. 748-756.
- Ha D., Dai A., Le Q. V. Hypernetworks //arXiv preprint arXiv:1609.09106. – 2016.
- Lorraine J., Duvenaud D. Stochastic hyperparameter optimization through hypernetworks //arXiv preprint arXiv:1802.09419. – 2018.
- Гребенькова О. С., Бахтеев О. Ю., Стрижов В. В. Вариационная оптимизация модели глубокого обучения с контролем сложности //Информатика и её применения. – 2021. – Т. 15. – №. 1. – С. 42-49.
- Finn, Chelsea, Pieter Abbeel, and Sergey Levine. "Model-agnostic meta-learning for fast adaptation of deep networks." International conference on machine learning. PMLR, 2017.
- Olga Grebenkova, Oleg Bakhteev, Vadim Strijov , Deep Learning Model Selection With Parametric Complexity Control , ICAART 2023.
- Dai, Damai, et al. "Why can gpt learn in-context? language models secretly perform gradient descent as meta optimizers." arXiv preprint arXiv:2212.10559 (2022).
- Scheme theorem: <https://engineering.purdue.edu/~sudhoff/ee630/Lecture03.pdf>
- Zoph B., Le Q. V. Neural architecture search with reinforcement learning //arXiv preprint arXiv:1611.01578. – 2016.
- Liu H., Simonyan K., Yang Y. Darts: Differentiable architecture search //arXiv preprint arXiv:1806.09055. – 2018.
- Pham H. et al. Efficient neural architecture search via parameters sharing //International conference on machine learning. – PMLR, 2018. – С. 4095-4104.
- Frankle J., Carbin M. The lottery ticket hypothesis: Finding sparse, trainable neural networks //arXiv preprint arXiv:1803.03635. – 2018.
- Zhou H. et al. Deconstructing lottery tickets: Zeros, signs, and the supermask //Advances in neural information processing systems. – 2019. – Т. 32.
- Gaier A., Ha D. Weight agnostic neural networks //Advances in neural information processing systems. – 2019. – Т. 32.
- Veniat T., Denoyer L. Learning time/memory-efficient deep architectures with budgeted super networks //Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. – 2018. – С. 3492-3500.
- Fernando C. et al. Evolution Channels Gradient Descent in Super Neural Networks //arXiv preprint arXiv:1701.08734. – 2017. – С. 1-16.
- Zhang, Yanbo, et al. "Diffusion models are evolutionary algorithms." arXiv preprint arXiv:2410.02543 (2024).
- Real, Esteban, et al. "Automl-zero: Evolving machine learning algorithms from scratch." International conference on machine learning. PMLR, 2020.